
NIGHT RAID
STUDIOS
CREATE. INNOVATE. CONQUER.
UNREAL ENGINE
unity
GODOT Game engine

Theater Ticketing System

Software Requirements Specification

Version 2.0

July 20, 2026

Group #9

Brylee Barsanti, Mark Jimenez

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Summer 2026

Revision History

Date	Description	Author	Comments
<date>	<Version 1>	<Your Name>	<First Revision>
07/14/26	1.0	Brylee, Mark	Initial Requirements Specification
07/20/26	2.0	Brylee, Mark	Added Software Design Specification
07/28/26	3.0	Byrlee, Mark	TEST CASE

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

1. Introduction

The purpose of this is to describe our software project, its main features, functionality, and design. It overviews how our system acts, its limitations, and how it is organized.

1.1 Purpose

- ***The purpose of this Software Requirements Specification is to define the requirements for the Theater Ticketing System. This document describes the system's intentions, functions, limitations, interfaces, and quality requirements. It will be used by the client and testing team as a guide for designing the system.***

1.2 Scope

- ***The Theater Ticketing System will allow people to search for movies, look at showtimes, purchase tickets, and receive digital or printable tickets through a web browser or a theater kiosk. Both access methods will use the same live ticket, pricing, theater, and showtime info.***

- ***The system will support: regular and deluxe theaters, customer discounts, optional customer accounts, loyalty memberships, payment processing, customer feedback, and management of the movies, theaters, and showtimes. The system will serve 20 theater locations in the San Diego area. Nearby-theater recommendations may be added in a later version.***

1.3 Definitions, Acronyms, and Abbreviations

SRS: *Software Requirements Specification*

Customer: *A person who searches for movies or is going to purchase tickets.*

Administrator: *An employee who manages theaters, showtimes, pricing, and customer purchase issues.*

Theater Employee: *An employee who helps customers at the movie theater and handles refunds or exchanges.*

Kiosk: *A browser-based ticketing system. Usually at the theater.*

Regular Theater: *A theater with 150 general admission seats.*

Deluxe Theater: *A theater with 75 assigned seats.*

Showtime: *The scheduled date and time of a movie showing.*

Loyalty Account: *An optional customer account that stores membership information, purchase history, and loyalty points.*

1.4 References

1. CS 250 Software Requirements Specification Template.
2. CS 250 lecture notes on requirements and use cases.
3. IEEE Recommended Practice for Software Requirements Specifications.

1.5 Overview

- ***Section 2 is a summary of the Theater Ticketing System, including its users, functions, assumptions, and dependencies.***
- ***Section 3 is more tailored to the system's interface requirements, functional requirements, use cases, and non-functional requirements.***

2. General Description

- This section gives a general overview of the project and the factors that affect how the software is designed and developed. It provides us with all of the background information, explains the overall purpose, and outlines any assumptions or limitations.

2.1 Product Perspective

- The Theater Ticketing System is intended to replace the client's current ticketing system, which is slow and has bugs. The new system will provide online and in-person access while using one shared database for all 20 San Diego theater locations. Ticket availability, prices, seats, and showtimes will update across the whole system.

2.2 Product Functions

.

- The system allows customers to search for movies, view showtimes, select tickets, select seats for deluxe theaters, apply discounts, process payments, generate tickets, create optional customer accounts, use loyalty memberships, collect customer feedback, manage showtimes, maintain purchase logs, and display the ratings of every movie.

2.3 User Characteristics

- Customers will use the system through a web browser or an in-theater kiosk. Customers are <u>expected</u> to have basic experience using websites.

- ***Theater employees will help customers and process refunds or exchanges in person.***

- ***Administrators will manage movies, theaters, showtimes, pricing, and customer purchase issues. Administrators will also require special login credentials and training on the management functions.***

- *External systems include the payment processor, email deliveries, central theater database, loyalty system, and movie review service.*

2.4 General Constraints

- *The initial prototype must be completed within four months. The project budget is $500,000. The system must run through a web. The system will support 20 theater locations in the San Diego area and will operate in Pacific Time.*
- *The system will use one shared database partitioned by theater. Updates should cause as little downtime as possible. Customers can purchase tickets no more than two weeks before a showing and no later than ten minutes after the showing begins. Each transaction is limited to 20 tickets.*

2.5 Assumptions and Dependencies

-The system assumes that customers have internet access or access to an in-theater kiosk. The system depends on the availability of the central database, payment processor, email service, and theater network. Theater employees are responsible for entering accurate movie, theater, price, and showtime information. Movie reviews and ratings depend on access to a third-party service. Printable tickets depend on a functioning theater or kiosk.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- **The system shall provide a browser-based user interface. The interface should support English, Spanish, and Swedish. Customers should be able to browse movies, showtimes, select tickets, enter payment information, and receive purchase confirmation. The interface will be available through the public website and in-theater kiosks. Both interfaces show the same current information. The system shall provide a separate authorized interface for administrators. The purchasing interface will display a five-minute timer after the customer begins too checkout.**

3.1.2 Hardware Interfaces

- **The system should interact with the following: theater kiosks, ticket printers, employee workstations, and customer devices that have a web browser.**

3.1.3 Software Interfaces

- **The system shall interface with the central theater database, payment services, email delivery service, loyalty account system, system logging service, and an approved movie review information source.**

3.1.4 Communications Interfaces

- **The system communicates through secure internet connections. Ticket purchases, seat reservations, pricing, and showtime changes are updated on the shared database in real time.**

3.2 Functional Requirements

FR-01 The system should allow customers to browse available movies.

FR-02 The system will display available showtimes for each movie and theater.

FR-03 The system allows customers to purchase tickets through a web browser.

FR-04 The system shall allow customers to purchase tickets through an in-theater kiosk.

FR-05 The website and kiosk use the same current ticket, price, seat, and showtime information.

FR-06 The system must prevent ticket sales more than two weeks before a showtime.

FR-07 The system will allow ticket sales until ten minutes after the scheduled showtime begins.

FR-08 The system is going to limit each transaction to no more than 20 tickets.

FR-09 The system shall support regular, student, military or veteran, and senior ticket pricing.

FR-10 The system will have on display the consistent ticket prices across all interfaces.

FR-11 The system should support regular theaters with a capacity of 150 customers.

FR-12 The system will support deluxe theaters with a capacity of 75 assigned seats.

FR-13 The system is going to allow customers to select available seats for deluxe showings.

FR-14 The system should treat regular showings as general admission.

FR-15 The system holds selected tickets or seats for five minutes after checkout

begins. FR-16 The system releases unpaid tickets or seats when the five-minute timer

expires. FR-17 The system prevents the sale of more tickets than the available theater

capacity. FR-18 The system can process customer payments.

FR-19 The system supports credit card, PayPal, and Bitcoin payments.

FR-20 The system generates a unique ticket after a successful purchase.

FR-21 The system prevents a ticket from being reused or duplicated.

FR-22 The system provides digital and printable tickets.

FR-23 The system provides tickets by email or through the kiosk ticket printer.

FR-24 The system lets customers create optional accounts.

FR-25 The system stores purchase history, loyalty points, and account information for registered customers.

FR-26 The system lets only one active device login per customer account.

FR-27 The system allows customers to purchase and use a loyalty membership.

FR-28 The system maintains daily logs of ticket purchases.

FR-29 The system allows administrators to add, edit, and remove showtimes.

FR-30 The system lets administrators change theater assignments.

FR-31 The system lets authorized administrators correct customer purchasing errors.

FR-32 The system requires refunds and exchanges to be completed by an authorized theater employee.

FR-33 The system collects optional customer feedback after a purchase.

FR-34 The system provides a waiting queue for high-demand showings.

FR-35 The system detecst and block suspicious automated ticket-purchasing behavior.

FR-36 The system displays approved movie ratings and critic quotes when available.

FR-37 The system records the number of tickets sold for each movie.

FR-38 The system shall allow administrators to view ticket sales and ticket prices for each movie.

3.2.1 <Functional Requirement or Feature #1>

3.2.1.1 Introduction
3.2.1.2 Inputs
3.2.1.3 Processing
3.2.1.4 Outputs
3.2.1.5 Error Handling

3.2.2 <Functional Requirement or Feature #2>

…

3.3 Use Cases

3.3.1 Use Case #1 Purchase Movie

Tickets Name: Purchase Movie Tickets

Brief Description:
A customer selects a movie, showtime, ticket type, and payment method to complete a ticket purchase.

Primary Actor:
Customer

Other Actors:
Payment Processor, Email Service

Preconditions:
The system is available.
The selected movie and showtime exist.
Tickets are available for the selected showing.

Basic Flow:
1. The customer selects a movie.
2. The system displays available theaters and showtimes.
3. The customer selects a showtime and ticket quantity.
4. The customer selects a ticket type and chooses seats if the showing is in a deluxe theater.
5. The system holds the selected tickets or seats for five minutes.
6. The customer selects a payment method and submits payment.
7. The system confirms payment and updates ticket availability.
8. The system issues a unique digital or printable ticket.
9. The system offers the customer an optional feedback rating.

Alternate Flows:
1. If the customer requests more than 20 tickets, the system asks the customer to reduce the quantity.
2. If the showing is sold out, the system informs the customer that tickets are unavailable.
3. If payment is declined, the system asks the customer to use another payment method.
4. If the five-minute timer expires, the system releases the held tickets or seats.
5. If suspicious automated behavior is detected, the system blocks or delays the transaction.
6. If the purchase is outside the allowed time period, the system rejects the purchase.

Postconditions:
A successful purchase creates a valid ticket and updates the available ticket or seat count.
An unsuccessful purchase does not make or print a ticket.

3.3.2 Use Case #2 Manage Showtime Name: Manage Showtimes

Brief Description:
An administrator creates, updates, or removes movie showtimes and theater assignments.

Primary Actor:
Administrator

Preconditions:
The administrator has valid login credentials.
The administrator is authorized to manage showtimes.

Basic Flow:
1. The administrator logs into the management interface.
2. The administrator selects the showtime management option.
3. The system displays current movies, theaters, and showtimes.
4. The administrator adds or edits a showing.
5. The administrator selects the movie, theater, date, and time.
6. The system checks for scheduling conflicts.
7. The administrator confirms the change.
8. The system updates the shared database.

Alternate Flows:
1. If the login credentials are invalid, the system denies access.
2. If the theater is already scheduled, the system displays a conflict.
3. If the selected theater type or capacity is invalid, the system rejects the change.
4. If existing purchases are affected, the system warns the administrator before saving.
5. If the database is unavailable, the system does not save the change.

Postconditions:
The selected showtime is added, updated, or removed from the shared database.

3.3.3 Use Case #3 Create and Use Loyatly

Account Name: Create and Use Loyalty Account

Brief Description:
A customer creates an optional account, activates a loyalty membership, and uses membership benefits during a ticket purchase.

Primary Actor:
Customer

Other Actor:
Payment Processor

Preconditions:
The customer has access to the website or kiosk.

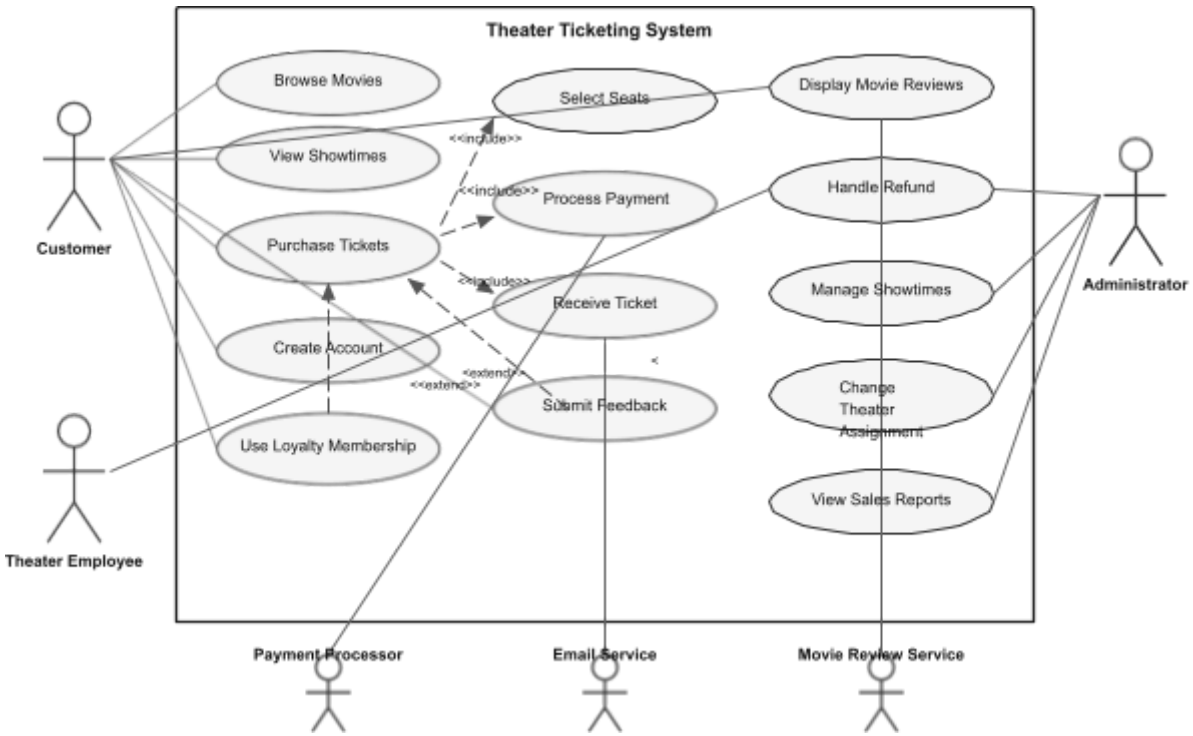
The customer has a valid email address.

Basic Flow:
1. The customer selects the option to create an account.
2. The customer enters the required personal information.
3. The system validates the entered information.
4. The customer selects and purchases a loyalty membership.
5. The system creates the account and activates the membership.
6. The customer logs into the account.
7. The customer selects a movie and showtime.
8. The system applies any eligible membership benefits.
9. The system records the purchase and updates the customer's loyalty points.

Alternate Flows:
1. If the email address is already registered, the system asks the customer to log in.
2. If membership payment fails, the system does not activate the membership.
3. If the login information is incorrect, the system denies access.
4. If the account is already active on another device, the system denies the second login.
5. If the membership has expired, the system asks the customer to renew it.

Postconditions:
The account and membership are active after a successful registration.
Eligible purchases and loyalty points are stored in the customer account.

Theater Ticketing System – UML Use Case Diagram



…

3.4 Classes / Objects

3.4.1 <Class / Object #1>

3.4.1.1 Attributes
3.4.1.2 Functions
<Reference to functional requirements and/or use cases>

3.4.2 <Class / Object #2>

…

3.5 Non-Functional Requirements

3.5.1 Performance

NFR-P1 The prototype shall support at least 1,000 simultaneous users.

NFR-P2 The system shall load normal movie and showtime pages within two seconds for at least 95 percent of requests under expected use.
NFR-P3 Successful purchases shall update ticket and seat availability within two seconds.

NFR-P4 The system shall use a waiting queue when demand exceeds safe processing capacity.

3.5.2 Reliability

NFR-R1 The system shall prevent two customers from purchasing the same assigned seat.

NFR-R2 A completed payment shall result in either a valid ticket or a recorded transaction error.

NFR-R3 The system shall preserve purchase records during temporary system interruptions.

NFR-R4 The system database shall be backed up daily.

3.5.3 Availability

NFR-A1 The system shall be available at least 99.5 percent of the time, excluding planned maintenance.

NFR-A2 Planned maintenance shall occur during low-demand periods.

NFR-A3 System updates shall be performed with minimal interruption to ticket sales.

3.5.4 Security

NFR-S1 Payment and account information shall be transmitted through encrypted connections.

NFR-S2 Customer passwords shall be securely hashed.

NFR-S3 Administrative functions shall require authorized credentials.

NFR-S4 Each ticket shall contain a unique validation identifier.

NFR-S5 The system shall detect suspicious automated purchase activity.

NFR-S6 Administrative changes shall be recorded in an audit log.

3.5.5 Maintainability

NFR-M1 Movie, theater, price, and showtime information shall be updateable without changing the main application source code.

NFR-M2 The system shall maintain error and transaction logs.

NFR-M3 The system shall include documentation for future maintenance and updates.

3.5.6 Portability

NFR-PT1 The customer interface shall operate in current versions of major web browsers.

NFR-PT2 The system shall not require a native mobile application.

NFR-PT3 The interface shall support desktop, tablet, kiosk, and mobile browser screen sizes.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

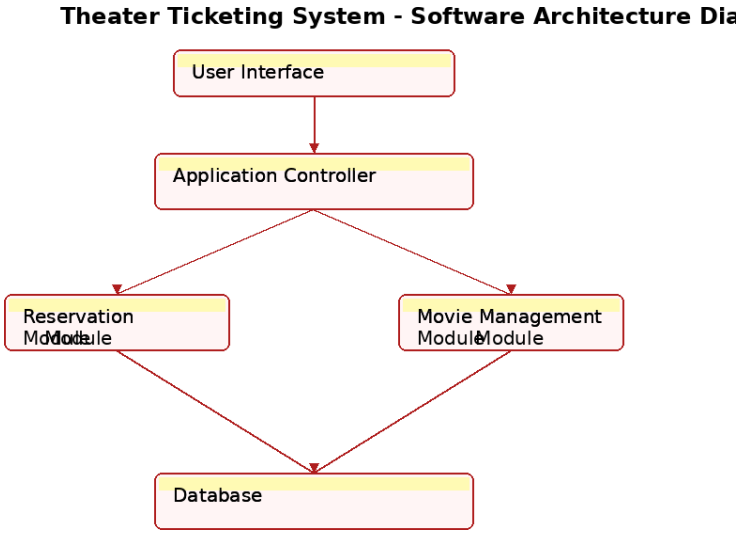
6. Software Design

6.1 System Overview

- The Theater Ticketing System is set up to make buying movie tickets easier for both the customers and theater employees that are selling them. Customers can look through all the movies, the different showtimes, purchase ttheir ickets online or through a kiosk that's at the theater, choose seats if they want to be in deluxe theaters, and receive either digital or printed tickets. The theater employees are there to help customers with movie ticket refunds or exchanges, while the administrators of the theater can manage and oversee the

movies, theaters, pricing, and showtimes. The system uses one shared database so every location always has the same up-to-date information at all times.

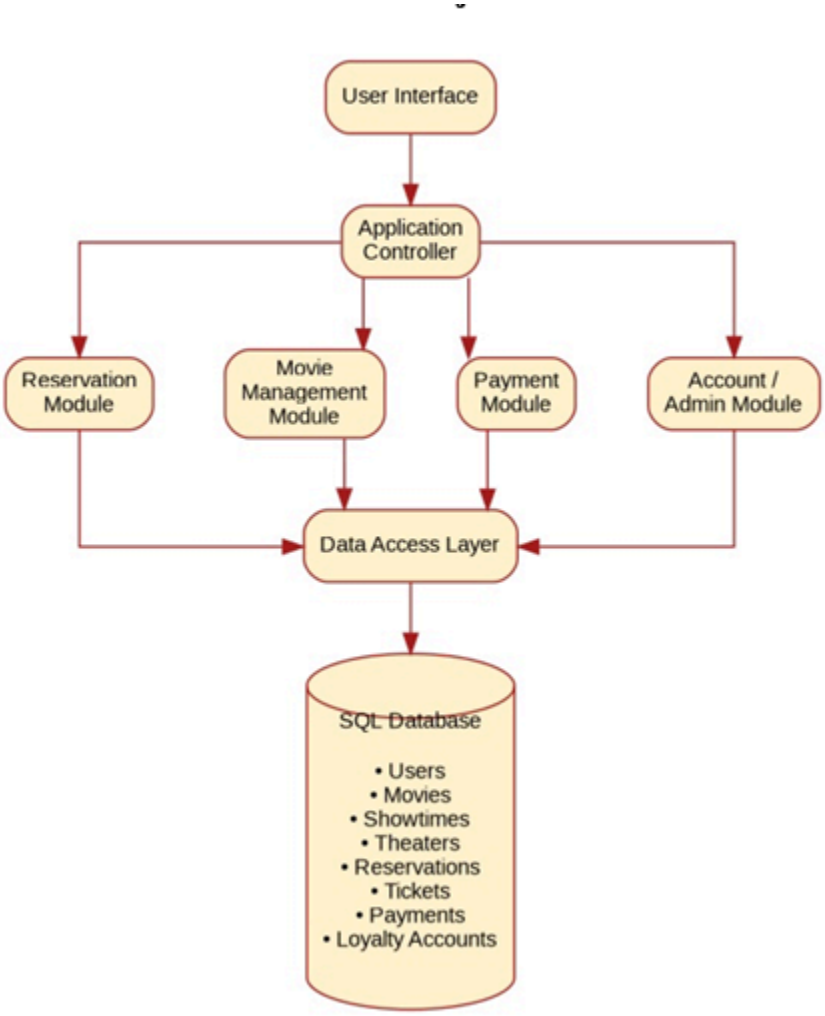
6.2 Software Architecture Overview



6.2.1 Architecture Description

- The software system follows a multi-layered architecture design. The customers interact with the system through the website or through the kiosk in which they buy tickets at.

Those requests are then sent to the application controller, which handles all the business side of things. Different things that are also included in this would be how they manage the reservations, movies, payments, customer accounts, and administration. All of this information is stored in a joint database.



Updated Software Architecture Diagram Above Compared to Assignment 2, we updated the architecture by adding a Data Access Layer and separate modules for payments and customer accounts. We also made the database bigger to show the main tables used by the system. These updates make it easier to understand and better to show how the different parts of the system work together.

(UPDATED RESPONSE BELOW) - The software system still follows a multi-layered system, but we updated it by adding a Data Access Layer. Customers use the system through the website or the kiosk, and their requests are sent to the application controller. Instead of every module connecting straight to the database, they all go through the Data Access Layer first. This keeps the system organized and makes it easier to update later. The SQL database stores all of the information so every theater has the same up-to-date stuff as needed.

Database Design The database is split into nine different tables based on what information each part of the system needs. Each table has its own unique ID, and the tables are connected so the system can keep track of everything without having to repeat information over and over. The main tables include:

- **-Customers**
- **-Movies**
- **-Showtimes**
- **-Theaters**
- **-Reservations**
- **-Tickets**
- **-Payments**
- **-Loyalty Accounts**
- **-Administrators**

6.2.3 Design Decisions & Tradeoffs

- Both of us decided to use one SQL database because it made the most sense for this project. Close to all of the information is connected, so having everything in one place makes it easier to manage and keeps everything updated.

- One option we thought about was using a NoSQL database like MongoDB. Which would work better for projects where the data changes a lot, but our system has a pretty organized structure, so SQL seemed like the better choice.

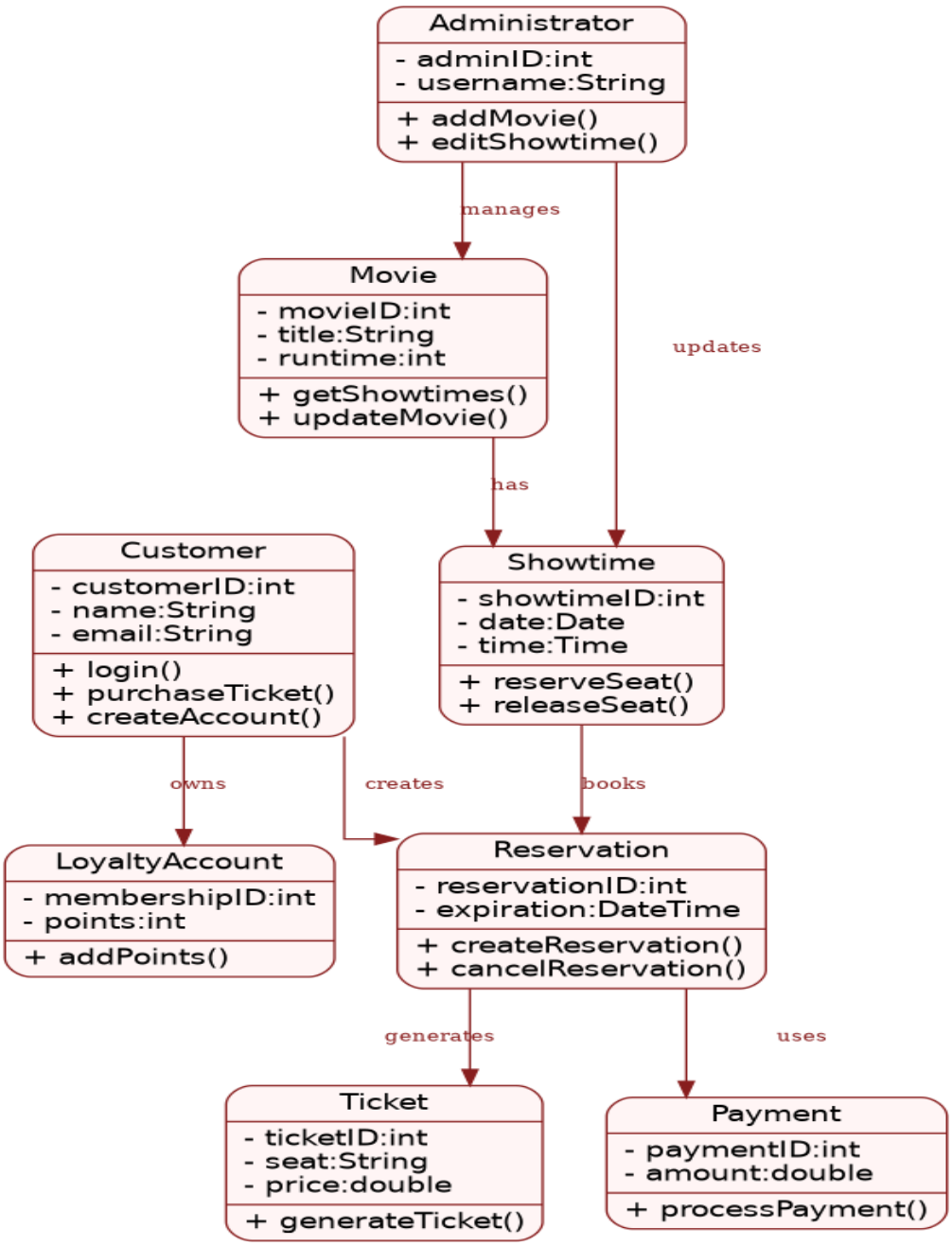
- Another option would have been splitting everything into multiple databases. For example, one database could store customer accounts while another stores reservations and payments. That might work better for a larger system, but it would also make everything more complicated since the databases would constantly need to communicate with each other.

- Using one shared database keeps the project simpler, and still gives us everything we need for the Theater Ticketing System.

- **Since our database will handle customers' sensitive data like payment information details and customer personal information we would encrypt all personal data using AES and TLS/SSL for data in transit. That way all personal information stays safe.**

6.3 UML Class Diagram

Figure 2. UML Class Diagram



6.4 Class Descriptions

Customer

Represents a person using the system to purchase a ticket or partake in a refund or exchange.

Attributes: - customerID : int, - name : String, - email : String, - password : String.

Operations: + login(email, password), + purchaseTicket(), + createAccount() + viewPurchaseHistory().

Movie: Stores all the information and descriptions about what each movie entails and what tickets and slot times are available for purchase.

Attributes: - movieID : int, - title : String, - runtime : int, - rating : String - description : String.

Operations: + getShowtimes(), + updateMovie().

Showtime: the scheduled showing information for a movie.

Attributes: - showtimeID : int, - date : Date, - time : Time, - seatsAvailable : int.

Operations: + reserveSeat(), + releaseSeat(), + updateAvailability().

Theater: Represents one theater location.

Attributes: - theaterID : int, - location : String, - theaterType : String - capacity : int.

Operations: + assignMovie(), + updateCapacity().

Ticket: Represents a purchased movie ticket.

Attributes: - ticketID : int, - seatNumber : String, - price : double - barcode: String.

Operations: + generateTicket(), + validateTicket().

Reservation: **Stores temporary ticket reservations during checkout.**

Attributes: - reservationID : int, - expirationTime : DateTime.

Operations: + createReservation(), + cancelReservation().

Payment: **Handles payment transactions.**

Attributes: - paymentID : int, - amount : double, - paymentMethod : String.

Operations: + processPayment(), + verifyPayment().

LoyaltyAccount: **customer loyalty information.**

Attributes: - membershipID : int, - points : int, - membershipStatus : String.

Operations: + addPoints(), + redeemPoints().

Administrator: **Allows authorized users to manage/control/operate the system.**

Attributes: - adminID : int, - username : String, - password : String

Operations: + addMovie(), + editShowtime(), + removeMovie(), + manageTheaters().

6.5 Development Plan

Week 1: **Finalize all the software requirements make sure it's meeting it.**

Week 2: **Design software architecture and the UML diagrams.**

Week 3: **Implement customer and reservation classes.**

Week 4: **Implement payment processing and all the administrator functions and needs.**

Week 5: **Complete testing and bug fixes.**

Week 6: **Finalize documents and prepare the final project.**

6.6 Team Member Responsibilities

Brylee Barsanti: Requirements documentation, UML diagrams, customer interface design, testing, final report

Mark Jimenez: Software architecture, class implementation, database design, payment module, GitHub management

Both can share every responsibility.

7.0 SDS Test Plan

- **This section layout all the test plans used for verification and validation of our theater ticketing system. Each test case below identifies a certain feature of the design.**

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
TC_MTS_001	Movie Search (Functional)	P1	Verify customers can search for an existing movie for the data base	Movies are loaded onto the web page from our library of movies currently showing. The movie searched for this test will be "Spider Man"	1. Write the url http://NRMovieTicekts.com in the URL browser and press enter 2. Once on the main page, our site will display the current movies being played in theaters the user must now select one. " Spider Man" 3.Movie is now selected displaying	The movie that the user has selected has been displayed correctly	The movie that the user has selected has been displayed correctly	PASS	Mark .J
TC_MTS_002	Showtime Selection (Functional)	P1	Verify the available showtimes are displayed correctly for selected movie	Selecting the current time the user wishes to watch the selected movie. Selected Movie will be "Spider Man" at 7:00pm	1. Select a movie ex "Spider Man" 2. View available showing times for "Spider Man" 3. Select July 24th 4. Time 7:00pm	All available shows are displayed correctly.	The moving showing for "Spider Man" was displayed correctly.	PASS	Brylee.B

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
TC_MTS_003	Seat Selection (Functional)	P1	Verify the available seats are displayed correctly for selected movie	Select seats for "Spider Man" at 7:00pm	1. Make sure seat map displays correct information 2. Select two seats "A4 & A5 " 3. Make sure the selected seats are confirmed and functional	All available seats will display correctly and will be shown to the user.	The seats display selection functions correctly displaying the seats of user selection	PASS	Mark .J
TC_MTS_004	Payment Screen (Functional)	P1	Verify payment method is running properly. After the following three test above are complete	Reaches payment method screen	1. Payment screen information is displayed 2. Valid Payment information is entered 3. Confirm payment 4. User receives confirmation on payment	Payment is processed correctly	The payment method was processed correctly	PASS	Brylee.B
TC_MTS_005	Ticket Conformation (Functional)	P1	Verify that after the payment is successful the customer receives ticket	Final screen after payment is successful	1. User complete payment screen 2. Ticket conformation right afterwards	After payment is successful, the ticket is displayed to the user	Ticket was displayed to user with all correct movie info	PASS	Mark .J
TC_MTS_006	Updates Seat Selection (System)	P2	Verify that once a seat has been reserved, it is no longer available to be purchased again for that specific time	Reserved seats cannot be reserved twice for same movie/showtime	1. Return to selected movie " Spider Man" 2. Make sure seat A5 & A6 are no longer available to be reserved	All reserved seats will no longer be available to be reserved twice	A5 & A6 Seats were no longer available, for reservation	PASS	Brylee.B
TC_MTS_007	Update Movie time	P2	Verify that once a showtime has already passed the 30mins marker, users are no longer able purchase tickets for the select movie "SpiderMan"	30min grace period to purchase ticket for x movie	1. Select "Spider Man" 2. July 23th 3. Make sure 7:00pm showtime is no longer available at 7:31pm	Movie no longer be available for purchase	Movie is no longer available for purchase	PASS	Mark .J
TC_MTS_008	Login Security (system)	P2	Verify the system allows access to users with entering correct information	Username: xxxx Password: xxxx	1. open Login page 2. Enter Username/Email 3. Enter Password 4.Login	If access is allowed, the user will see his information along with ticket purchases and payment information.	If access is allowed, the user will see his information along with ticket purchases and payment information.	PASS	Brylee.B

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
TC_MTS_009	UI Check (system)	P3	Verify UI is user friendly, allowing for easy navigation and all major feature function	Website loads all correct functions	1. Open Brave Browser 2.Navigate Movie tab 3. select movie 4. Return / repeat for other movie	All pages load correctly	Navigation through all tabs worked correctly and load properly displaying all correct information	PASS	Mark .J
TC_MTS_010	Cancellation Ticket (function)	P2	Verify once user reaches payment screen, user is allowed to cancel and backtrack towards showtime/Movie Name	Customer has selected Spider Man at 7:00pm but chose the wrong seats, wants to backtrack to select correct seats (This test will also function if user wants to select new movie, new showtime , new date etc...)	1. Select Movie "Spider Man" 2. July 23th 3.7:00pm 4.Seats A5 & A6 5. Cancel Purchase 6. Returned to home screen	Purchase was canceled, should be returned to home page	Purchase was successfully canceled, the user was redirected back to the starting screen.	PASS	Bryle e.B
TC_MTS_011	Login with incorrect information (function)	P1	Enter incorrect information such as password or username	Password1234!	1. User is on homepage 2.User clicks log in 3. User enters password	displays "Username/passowrd is incorrect"	If users is denied the system will display " Incorrect email or password try again" and will loop until user enters correct information	Fails (Should fail since its the incorrect information)	Mark .J
TC_MTS_012	Search for nonreconizable movie	P2	Enters an incorrect movieID that is not in our database	movieID is searched incorrectly	1. Enters MovieID in search bar	displays blank page with message	Blank page is displayed along with "Movie Doesn't Exist"	Fails (Should fail since its the incorrect information)	Bryle e.B
TC_MTS_013	Expired Information	P2	Form of payment method has been expired such as debit cards, credit cards	Customers inputs payment information but expiration date has passed	1. Complete all others tasks to reach payment screen 2. inputs payment information	displays payment screen notification "Payment for has expired, try new form"	displays payment screen notification "Payment for has expired, try new form"	Fails (Should fail since its the incorrect information)	Mark .J

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2